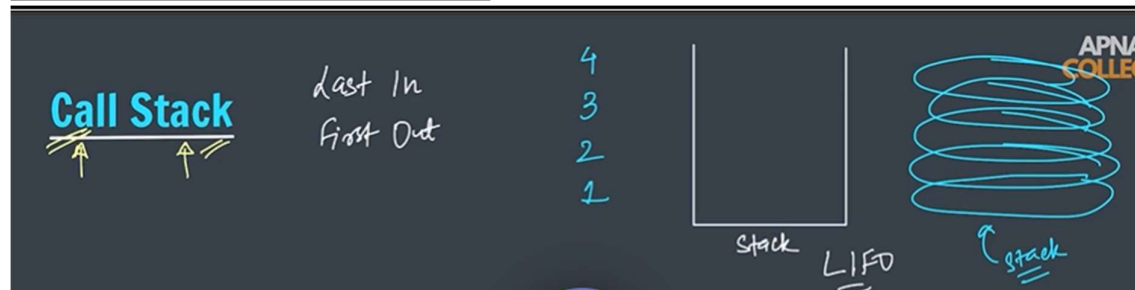


→JS-11

Call

```
app.js > ...
1 function hello() {
2   console.log("hello");
3 }
4 function hello(): void
5 hello();
6
```



→example code:

```
app.js > ...
1 function hello() {
2   console.log("hello");
3 }
4
5 function demo() {
6   hello();
7 }
8
9 demo();
```

The screenshot shows the Chrome DevTools Console. The "Elements" tab is selected, and the "No Issues" message is displayed. The "hello" function is highlighted in the console log.

```
app.js > ...
1 function hello() {
2   console.log("inside hello fnx");
3   console.log("hello");
4 }
5
6 function demo() {
7   console.log("calling hello fnx");
8   hello();
9 }
10
11 console.log("calling demo fnx");
12 demo();
13 console.log("done, bye!");
```

The screenshot shows the Chrome DevTools Console. The "No Issues" message is displayed. The console log shows the following output: "calling demo fnx", "calling hello fnx", "inside hello fnx", "hello", and "done, bye!".

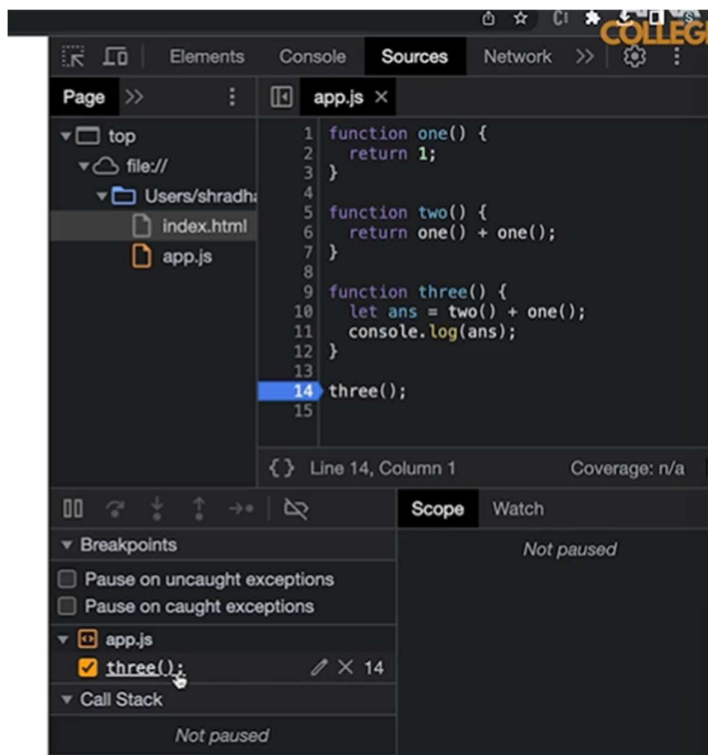
→ Visualizing the call Stack

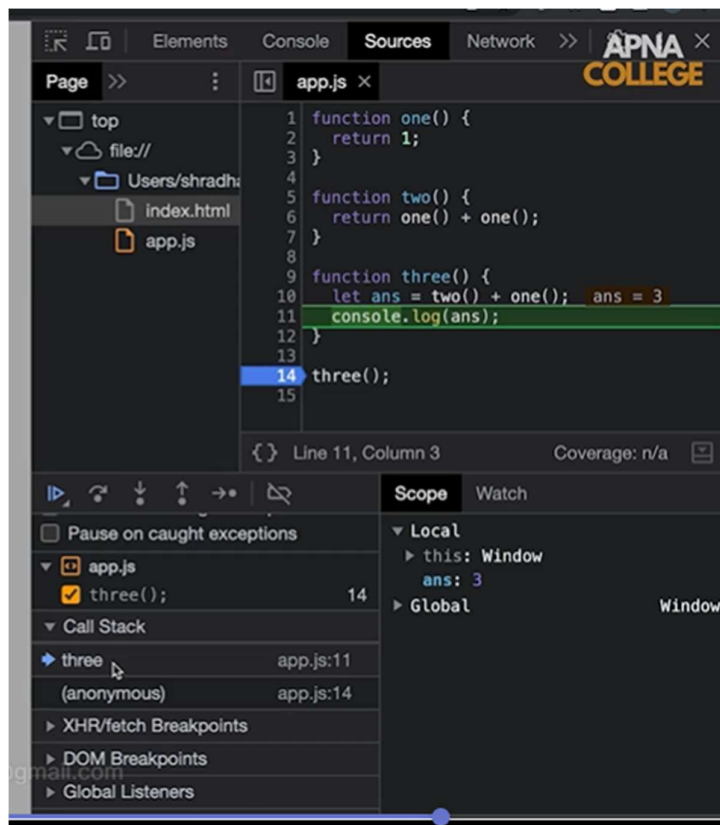
Visualizing the Call Stack

```
function one() {  
  return 1;  
}  
  
function two() {  
  return one() + one();  
}  
  
function three() {  
  let ans = two() + one();  
  console.log(ans);  
}
```

→ BreakPoints:

It is generally used for the debugging process





→ Js is single threaded

JS is Single Threaded

```

setTimeout(function () {
  console.log("apna college");
}, 2000);

console.log("hello ...");

```

```

app.js > ...
1 let a = 25;
2 console.log(a);
3 let b = 10;
4 console.log(b);
5 console.log(a + b);
6

```

No Issues

25

10

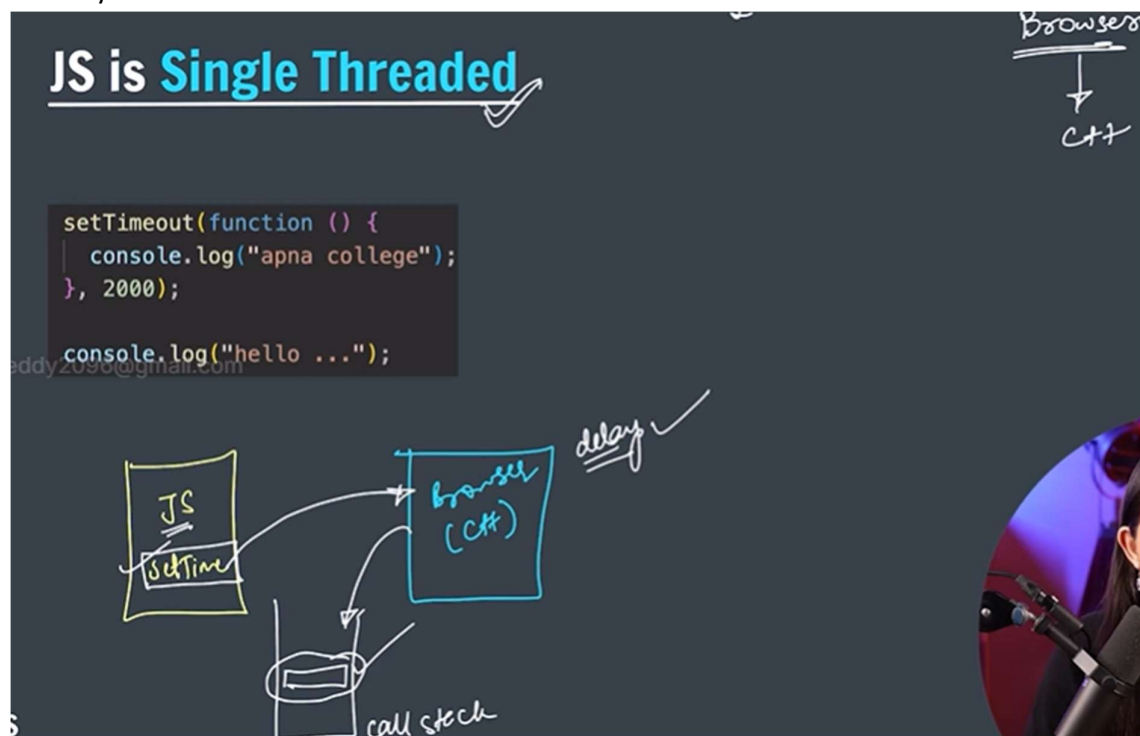
35

>

```
// console.log(a + b);  
  
setTimeout(() => {  
  console.log("apna college");  
}, 2000);  
  
console.log("hello...");  
  
// console.log(a + b);  
  
setTimeout(() => {  
  console.log("apna college");  
}, 2000);  
setTimeout(() => {  
  console.log("hello world");  
}, 2000);  
  
console.log("hello...");
```

The first screenshot shows the console output: "hello...", "apna college". The second screenshot shows the console output: "hello...", "apna college", "hello world".

→ here how it do the two works at a time when js is single threaded but this setTimeout is done by the browser



→ Synchronous means the normal flow of program when we use the setTimeout functions then it is asynchronous

→ Callback hell

```
h1 = document.querySelector("h1");

setTimeout(() => {
  h1.style.color = "red";
}, 1000);
```

Apna College

```
app.js > ...
h1 = document.querySelector("h1");

setTimeout(() => {
  h1.style.color = "red";
}, 1000);

setTimeout(() => {
  h1.style.color = "orange";
}, 2000);
```

Apna College

```
h1 = document.querySelector("h1");

setTimeout(() => {
  h1.style.color = "red";
}, 1000);

setTimeout(() => {
  h1.style.color = "orange";
}, 2000);

setTimeout(() => {
  h1.style.color = "green";
}, 3000);
```

Apna College

```
h1 = document.querySelector("h1");

function changeColor(color, delay) {
  setTimeout(() => {
    h1.style.color = color;
  }, delay);
}

changeColor("red", 1000);
changeColor("orange", 2000);
changeColor("green", 3000);
```

```

h1 = document.querySelector("h1");

function changeColor(color, delay, nextColorChange) {
  setTimeout(() => {
    h1.style.color = color;
    if (nextColorChange) nextColorChange();
  }, delay);
}

changeColor("red", 1000, () => {
  changeColor("orange", 1000);
});

```

```

js app.js > changeColor("red") callback > changeColor("orange") callb
1  h1 = document.querySelector("h1");
2
3  function changeColor(color, delay, nextColorChange) {
4    setTimeout(() => {
5      h1.style.color = color;
6      if (nextColorChange) nextColorChange();
7    }, delay);
8  }
9
10 changeColor("red", 1000, () => {
11   changeColor("orange", 1000, () => {
12     changeColor("green", 1000, () => {
13       changeColor("yellow", 1000);
14     });
15   });
16 });
17

```

Apna College

```

pp.js > ...
h1 = document.querySelector("h1");

function changeColor(color, delay, nextColorChange) {
  setTimeout(() => {
    h1.style.color = color;
    if (nextColorChange) nextColorChange();
  }, delay);
}

changeColor("red", 1000, () => {
  changeColor("orange", 1000, () => {
    changeColor("green", 1000, () => {
      changeColor("yellow", 1000, () => {
        changeColor("blue", 1000);
      });
    });
  });
});

//callbacks nesting -> callback hell

```

```
function savetoDb(data) {
  let internetSpeed = Math.floor(Math.random() * 10) + 1;
  if (internetSpeed > 4) {
    console.log("your data was saved : ", data);
  } else {
    console.log("weak connection. data not saved");
  }
}
```

```
No Issues
your data was saved : apna college app.js
< undefined
> savetoDb("apna college");
weak connection. data not saved app.js
< undefined
> savetoDb("apna college");
your data was saved : apna college app.js
< undefined
> savetoDb("apna college");
your data was saved : apna college app.js
< undefined
> savetoDb("apna college");
your data was saved : apna college app.js
< undefined
> savetoDb("apna college");
your data was saved : apna college app.js
< undefined
> savetoDb("apna college");
weak connection. data not saved app.js
```

```
function savetoDb(data, success, failure) {
  let internetSpeed = Math.floor(Math.random() * 10) + 1;
  if (internetSpeed > 4) {
    success();
  } else {
    failure();
  }
}

savetoDb(
  "apna college",
  () => {
    console.log("success : your data was saved");
  },
  () => {
    console.log("failure: weak connection. data not saved");
  }
);
```

```
No Issues
failure: weak connection. data not saved
>
```

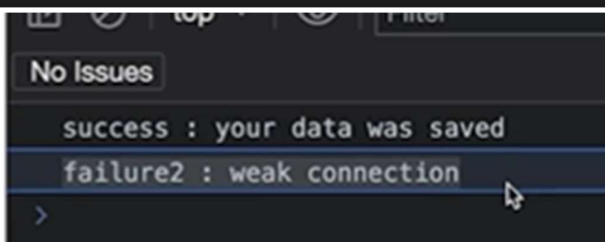


```

function saveToDb(data, success, failure) {
  let internetSpeed = Math.floor(Math.random() * 10) + 1;
  if (internetSpeed > 4) {
    success();
  } else {
    failure();
  }
}

saveToDb(
  "apna college",
  () => {
    console.log("success : your data was saved");
    saveToDb(
      "hello world",
      () => {
        console.log("success2: data2 saved");
      },
      () => {
        console.log("failure2 : weak connection");
      }
    );
  },
  () => {
    console.log("failure: weak connection. data not saved");
  }
);

```



```

saveToDb(
  "apna college",
  () => {
    console.log("success : your data was saved");
    saveToDb(
      "hello world",
      () => {
        console.log("success2: data2 saved");
        saveToDb(
          "shradha",
          () => {
            console.log("success3: data3 saved");
          },
          () => {
            console.log("failure3 : weak connection");
          }
        );
      },
      () => {
        console.log("failure2 : weak connection");
      }
    );
  },
  () => {
    console.log("failure: weak connection. data not saved");
  }
);

```

jcreddy2096@gmail.com


```

No issues
success : your data was saved
success2: data2 saved
failure3 : weak connection
> |

```

→ Promises

Promises

The Promise object represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

Promises → object

resolve & reject

↓ success ↓ failure

Promises → object

resolve & reject

↓ success ↓ failure

function (asynchronous)

State → pending ✓
rejected (error)
fulfilled (resolved)

```

function savetoDb(data) {
  return new Promise((resolve, reject) => {
    let internetSpeed = Math.floor(Math.random() * 10) + 1;
    if (internetSpeed > 4) {
      success("success : data was saved");
    } else {
      failure("failure : weak connection");
    }
  });
}

// savetoDb("apna college");

```

```
function savetoDb(data) {
  return new Promise((resolve, reject) => {
    let internetSpeed = Math.floor(Math.random() * 10) + 1;
    if (internetSpeed > 4) {
      resolve("success : data was saved");
    } else {
      reject("failure : weak connection");
    }
  });
}

// savetoDb("apna college");
```

```
top Filter Default levels
No Issues
> savetoDb("apna college");
< ▾ Promise {<fulfilled>: 'success : data was saved'}
  > [[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
    [[PromiseResult]]: "success : data was saved"
> savetoDb("apna college");
< > Promise {<fulfilled>: 'success : data was saved'}
> savetoDb("apna college");
< > Promise {<fulfilled>: 'success : data was saved'}
> |
```

Promises

then() & catch()

→ methods

Promise
 ↓
 fulfilled → .then
 ↓
 reject → error
 ↓
 .catch

```
let request = saveToDBPromise("apnacollege");
request
  .then(() => {
    console.log("promise resolved");
  })
  .catch(() => {
    console.log("promise rejected");
  });
```

```

function savetoDb(data) {
  return new Promise((resolve, reject) => {
    let internetSpeed = Math.floor(Math.random() * 10) + 1;
    if (internetSpeed > 4) {
      resolve("success : data was saved");
    } else {
      reject("failure : weak connection");
    }
  });
}

let request = savetoDb("apna college"); //req = promise object
request.then(() => {
  console.log("promise was resolved");
})
.catch(() => {
  console.log("promise was rejected");
})

```

```

No Issues
promise was rejected
>

```

```

6
7 function savetoDb(data) {
8   return new Promise((resolve, reject) => {
9     let internetSpeed = Math.floor(Math.random() * 10) + 1;
10    if (internetSpeed > 4) {
11      resolve("success : data was saved");
12    } else {
13      reject("failure : weak connection");
14    }
15  });
16 }
17
18 let request = savetoDb("apna college"); //req = promise object
19 request
20   .then(() => {
21     console.log("promise was resolved");
22     console.log(request);
23   })
24   .catch(() => {
25     console.log("promise was rejected");
26     console.log(request);
27   })

```

```

No Issues
promise was resolved app.js:71
promise was rejected app.js:72
▼ Promise {<fulfilled>: 'success : data was saved'}
  ► [[Prototype]]: Promise
  [[PromiseState]]: "fulfilled"
  [[PromiseResult]]: "success : data was saved"
>

```

```

savetoDb("apna college")
  .then(() => {
    console.log("promise was resolved");
  })
  .catch(() => {
    console.log("promise was rejected");
  });

```

```

No Issues
promise was resolved
>

```

→ Promise chaining

Promises

Improved Version

```
saveToDBPromise("apnacollege")
  .then(() => {
    console.log("promise1 resolved");
    return saveToDBPromise("hello world");
  })
  .then(() => {
    console.log("promise2 resolved");
  })
  .catch(() => {
    console.log("some promise rejected");
  });
```

Promise chaining ← callback

1 ✓
↓
2 ✓
↓
3 ✓

```
savetoDb("apna college")
  .then(() => {
    console.log("data1 saved");
    return savetoDb("helloworld");
  })
  .then(() => {
    console.log("data2 saved");
  })
  .catch(() => {
    console.log("promise was rejected");
  });
```

No Issues

data1 saved

data2 saved

>

```
savetoDb("apna college")
  .then(() => {
    console.log("data1 saved");
    return savetoDb("helloworld");
  })
  .then(() => {
    console.log("data2 saved");
    return savetoDb("shradha");
  })
  .then(() => {
    console.log("data3 saved");
  })
  .catch(() => {
    console.log("promise was rejected");
  });
```

No Issues

data1 saved app.js:70

data2 saved app.js:74

promise was rejected app.js:81

>

→ Results and errors in promises

Promises

promises are rejected and resolved with some data (valid results or errors)

```
saveToDBPromise("apnacollege")
  .then((result) => {
    console.log("result : ", result);
    console.log("promise1 resolved");
    return saveToDBPromise("hello world");
  })
  .then((result) => {
    console.log("result : ", result);
    console.log("promise2 resolved");
  })
  .catch((error) => {
    console.log("error : ", error);
    console.log("some promise rejected");
  });
```

```
savetoDb("apna college")
  .then((result) => {
    console.log("data1 saved");
    console.log("result of promise: ", result);
    return savetoDb("helloworld");
  })
  .then((result) => {
    console.log("data2 saved");
    console.log("result of promise: ", result);
    return savetoDb("shradha");
  })
  .then((result) => {
    console.log("data3 saved");
    console.log("result of promise: ", result);
  })
  .catch((error) => {
    console.log("promise was rejected");
    console.log("error of promise: ", error);
  });
```

No Issues

data1 saved	app.js:70
result of promise: success : data was saved	app.js:71
data2 saved	app.js:75
result of promise: success : data was saved	app.js:76
data3 saved	app.js:80
result of promise: success : data was saved	app.js:81

>

```
h1 = document.querySelector("h1");

function changeColor(color, delay) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      h1.style.color = color;
      resolve("color changed!");
    }, delay);
  });
}

changeColor("red", 1000)
  .then(() => {
    console.log("red color was completed");
    return changeColor("orange", 1000);
  })
  .then(() => {
    console.log("orange color was completed");
    return changeColor("green", 1000);
  })
  .then(() => {
    console.log("green color was completed");
    return changeColor("blue", 1000);
  })
  .then(() => {
    console.log("blue color was completed");
  });
```

Apna College

